

Intro to Tracing AI/ML Models with THAPI

Readable kernel identities + semantic model regions

Two focused demos show how specialization-aware Triton names and application-level ITT regions make THAPI/iprof timelines easier to navigate.

August 3, 2026 · 10:15 a.m. · [Official agenda](#)

Nathan S. Nichols

Start with the debugging question

- A training step is slower than expected. The timeline contains thousands of runtime events and device kernels.
- To explain the slowdown, we need three coordinates: **where in the model, what happened in the runtime, and which kernel specialization ran.**
- A trace preserves order and overlap—but only the application can supply its semantic vocabulary.

Goal: turn “something was slow” into a falsifiable statement about a step, phase, runtime event, and kernel.

REPEATED DDP RHYTHM



Compute, communication, and waiting share one time axis.

BY THE END, YOU CAN

A trace can be accurate—and still underspecified

KERNEL AMBIGUITY

Triton specializations may differ in block size, warps, or stages, while default labels may omit the choices we want to compare.

OPAQUE MODEL PHASES

Without application markers, the timeline shows runtime activity and kernels—not necessarily **initialization, forward, backward, or per-layer structure**.

SCALE MULTIPLIES CONFUSION

Distributed runs add ranks, collectives, and overlap. A trace with weak labels becomes hard to interpret precisely enough for debugging.

One mental model: add context at three levels

1. DISTRIBUTED RUNTIME / COMMUNICATION

Which step is waiting on synchronization? Where are collectives and runtime events happening across ranks?

2. MODEL PHASES / SEMANTIC REGIONS

Is time spent in initialization, data generation, forward, backward, optimizer step, or a specific layer block?

3. KERNEL-LEVEL SPECIALIZATION

Which concrete Triton specialization ran there? Was it the block-size / warps / stages variant we expected?

THE TALK IN ONE SENTENCE

ITT supplies model-phase context, **Triton specialization names** supply kernel identity, and **THAPI/iprof** captures supported application/runtime/device events on a timeline.

The exercises demonstrate the model and kernel layers separately. In a workload that

The capture path: workload → THAPI/iprof → Perfetto

WHY A TIMELINE?

- Timing summaries tell you **how much** time was spent, but traces tell you **when** things happened and **what overlapped**.
- For distributed AI, that lets you inspect **compute**, **communication**, and **waiting** on one time axis.
- Application annotations add the semantic scopes needed to navigate that activity.

THAPI IN THIS STORY

- **THAPI** is a portable tracing infrastructure for heterogeneous applications.
- `iprof` is its user-facing wrapper for capture and analysis; `-l` writes a Perfetto trace.
- **Perfetto** is the viewer and SQL analysis environment used in the exercises.

run workload



iprof capture



.pfttrace



Perfetto

What is Triton?

LANGUAGE + COMPILER

Triton provides a Python-based environment for writing custom, high-throughput GPU compute kernels—especially deep-learning primitives.

- A **kernel** describes one parallel device operation, not the whole model.
- Programs operate on blocks of values; the compiler maps that blocked program to GPU execution.
- Compile-time constants and launch choices create concrete **specializations**.

Python launch → Triton JIT → specialized kernel → GPU

Profiling question: which concrete specialization did that launch select?

Source: [Triton documentation](#)

Triton gap: the launch choices are performance context

- Triton often generates multiple **kernel specializations** for the same logical operation.
- Those specializations can differ in performance-relevant knobs like `BLOCK_SIZE`, `num_warps`, and `num_stages`.
- If the trace or profiler view does not expose a helpful name, it is hard to tell **which specialization** was active at a given point in time.

For performance debugging, “a Triton kernel ran here” is not enough; we want a name that preserves the specialization choice.

logical op: **vector add**

specializations:

- `BLOCK_SIZE=128, warps=4, stages=2`
- `BLOCK_SIZE=256, warps=8, stages=2`

risk: the labels omit the launch choices

better: each gets its own readable specialization name

Triton solution: give each specialization a readable name

```
def _vadd_repr(proxy):  
    bs = proxy.constants["BLOCK_SIZE"]  
    w = proxy.constants["W_NAME"]  
    s = proxy.constants["S_NAME"]  
    return f"vadd_bs{bs}_w{w}_s{s}"  
  
@triton.jit(repr=_vadd_repr)  
def vadd(X_ptr, Y_ptr, Z_ptr, N,  
        BLOCK_SIZE: tl.constexpr,  
        W_NAME: tl.constexpr,  
        S_NAME: tl.constexpr):  
    ...
```

This is a lightweight trick: keep the kernel logic the same, but make the specialization visible to the profiler.

- `@triton.jit(repr=_vadd_repr)` lets the kernel provide a **custom specialization-dependent representation**.
- The naming-only `constexpr` fields `W_NAME` and `S_NAME` let the name encode warps and stages even though those are launch knobs.
- The callback reads the specialization constants through `proxy.constants` and builds a stable string.

Triton example: launch site and resulting names

```
grid = (triton.cdiv(N, block_size),)

vadd[grid](x, y, z, N,
           BLOCK_SIZE=block_size,
           W_NAME=num_warps,
           S_NAME=num_stages,
           num_warps=num_warps,
           num_stages=num_stages)
```

Launch code from `run_once(...)` in the supplied example.

BLOCK_SIZE=128, warps=4, stages=2 → `vadd_bs128_w4_s2`

BLOCK_SIZE=256, warps=8, stages=2 → `vadd_bs256_w8_s2`

- Same logical kernel, but now each specialization has a **distinct profiler identity**.
- This helps with debugging, attribution, and cross-checking that the expected launch configuration actually appeared.
- The demo launches each configuration 20 times, verifies the result, and makes both names easy to find.

Demo checkpoint: make kernel variants visible

```
(  
  cd triton_vector_add_example_with_named_kernels  
  python test_vadd.py --device xpu  
  
  iprof -l vadd.pftrace \  
    --trace-output vadd-ctf -- \  
    python test_vadd.py --device xpu  
)
```

CHECKPOINT

1. Predict the name for `--config 512,8,3`.
2. Confirm the two defaults print as `vadd_bs128_w4_s2` and `vadd_bs256_w8_s2`.
3. Open the trace and search for `vadd_`.
4. Optional text check: `iprof -r vadd-ctf -k`.

Correctness is checked against PyTorch before each “verified” line is printed.

What is ITT?

INSTRUMENTATION API

Instrumentation and Tracing Technology (ITT) is an open-source profiling API for adding application-defined trace data.

- A **domain** namespaces one logical workload.
- A **task** brackets a named unit of work on a thread; tasks can nest like scopes.
- The Python binding exposes the context manager used here: `with ittapi.task(...)`.

Sources: [ITT API overview](#) · [Task API](#)

```
TrainingLoop
```

```
  Step.1
```

```
    Data.Generate
```

```
    Forward
```

```
      Layer.0
```

```
    Backward
```

```
    Optimizer.Step
```

Controlled comparison: same workload, one tracing switch

SHARED TRAINING PATH

llama3_demo_common.py

```
for step in range(args.steps):
    with regions.task(f"Step.{step}"):
        x, y = get_batch()
        with regions.task("Forward"):
            logits = model(x); loss = loss_fn(...)
        with regions.task("Backward"):
            loss.backward()
        with regions.task("Optimizer.Step"):
            optimizer.step()
```

ONLY THE REGION FACTORY CHANGES

Two thin entry points

```
# baseline: Regions.task() returns nullcontext()
run()

# instrumented: Regions.task() returns an ITT task
def itt_task(name, domain):
    return ittapi.task(name, domain=domain)

run(task_factory=itt_task)
```

Model architecture, seeds, inputs, optimizer, and control flow are shared. The baseline emits no application regions; the ITT entry point emits the names already present in the common path.

ITT granularity inside the model

```
def forward(self, x):  
    i = self.layer_index  
    with self.regions.task(f"Layer.{i}"):   
        with self.regions.task(f"Attn.{i}"):   
            x = x + self.attention(self.norm1(x))  
        with self.regions.task(f"MLP.{i}"):   
            x = x + self.mlp(self.norm2(x))  
    return x
```

Step.1

Data.Generate

Forward

Layer.0

Attn.0

MLP.0

Backward

Optimizer.Step

Default path. With `--checkpoint`, layer work is replayed during Backward.

Demo checkpoint: capture a controlled before / after

```
module load thapi
module load frameworks

python -c 'import torch, ittapi'

CCL_WORKER_AFFINITY=5 NPROCS=1 PPN=1 \
  iprof_demo_llama3/run_aurora_demo.sh \
  --mode both --steps 2 -- \
  --batch-size 1 --seq-len 16 --dim 64 \
  --layers 1 --heads 4 --vocab-size 128 --bf16
```

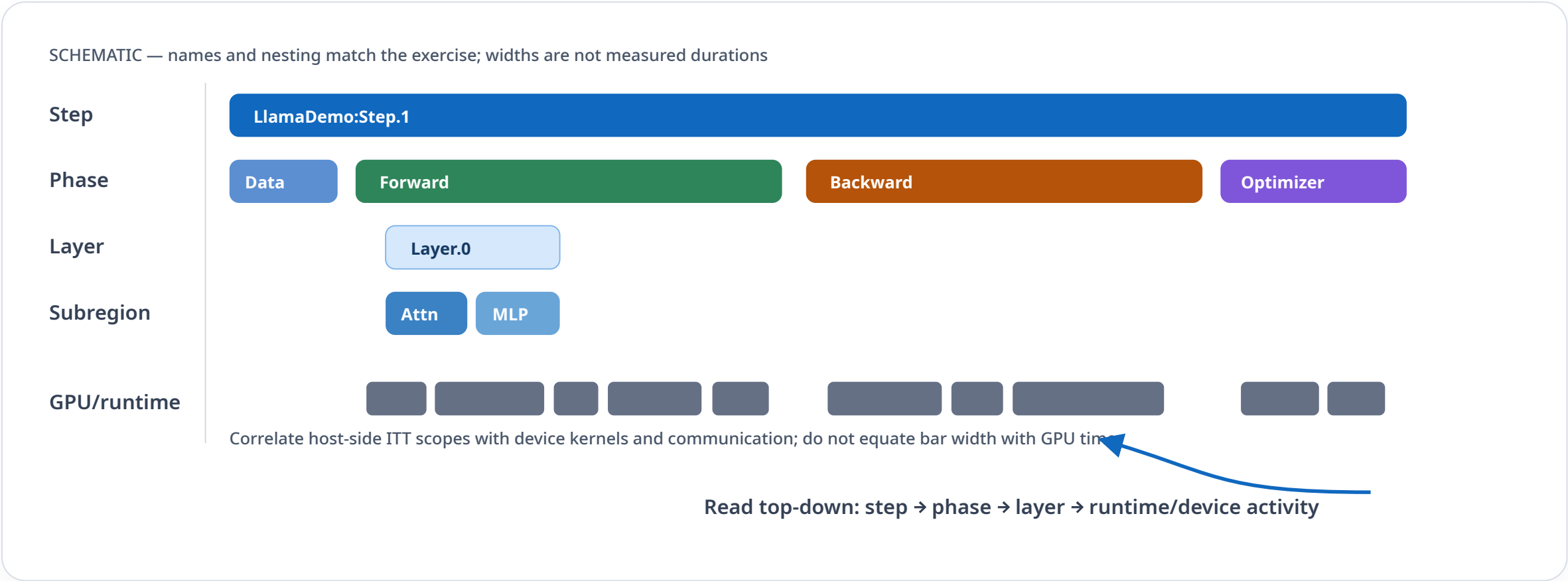
NO ALLOCATION? USE THE INCLUDED TRACES

- `traces/tiny_baseline.pftrace` — no `LlamaDemo:*` regions
- `traces/tiny_with_itt.pftrace` — 13 semantic names across two steps

```
python iprof_demo_llama3/inspect_trace_markers.py \
  iprof_demo_llama3/traces/tiny_baseline.pftrace \
  iprof_demo_llama3/traces/tiny_with_itt.pftrace
```

The launcher defaults to tracing rank 0; set `TRACED_RANKS=-1` for every rank.

Read the Llama trace top-down



1 • Locate

Open `tiny_with_itt.pftrace` and search for `LlamaDemo:Step.1`.

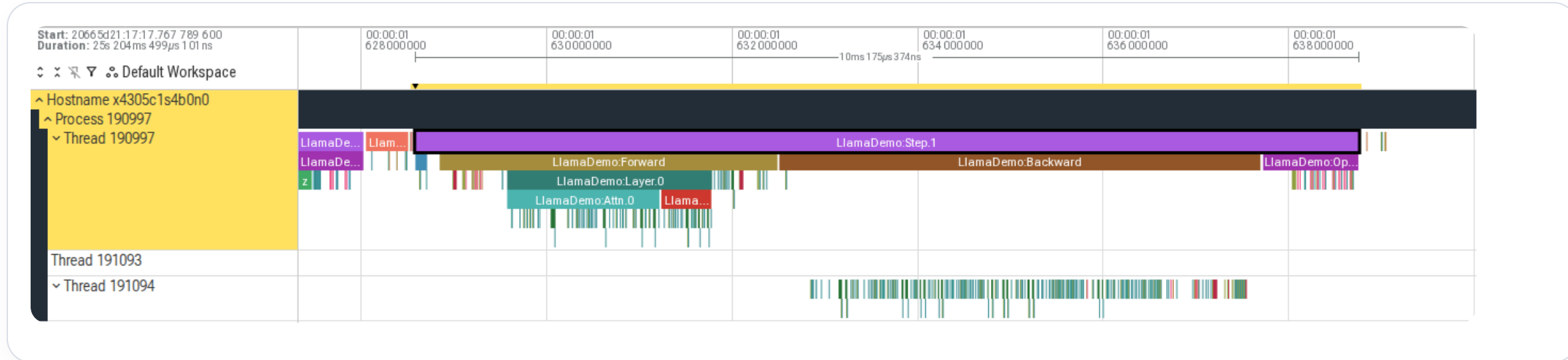
2 • Zoom

Follow `Forward → Layer.0 → Attn.0 / MLP.0`, then compare Backward and Optimizer.

3 • Correlate

Inspect device/runtime tracks at the same timestamps; run one query from `perfetto_queries.sql`.

Interactive result: explore the controlled trace



Static fallback from `tiny_with_itt.pftrace`; the browser presentation autloads the same trace into an interactive Perfetto view.

Outer envelope

`Step.1` provides one navigable unit of training.

Nested meaning

Forward, Layer.0, Attn.0, Backward, and Optimizer remain distinguishable.

Control

The matching baseline trace has the same workload but no `LlamaDemo:*` labels.

Two demos, one labeling strategy

Signal	What it tells you	Example question it answers
ITT regions	Model-phase and model-structure context	Was the slowdown in initialization, forward, backward, optimizer step, or a specific layer?
Triton specialization names	Which concrete kernel variant ran	Was this the <code>bs128_w4_s2</code> version or the <code>bs256_w8_s2</code> one?
THAPI/iprof timeline	Time alignment between those signals and runtime activity	In a workload using both: when did that specialization appear inside the model step?

Demonstrated separately here; composable in one workload. ITT gives semantic location, kernel names give identity, and THAPI/iprof provides time alignment for supported events.

Tips & guidance

TIPS & GUIDANCE

- Keep Triton names **stable, short, and meaningful**.
- Keep naming-only fields synchronized with the real launch options.
- Instrument the **major semantic regions first**; add detail only where it improves interpretation.
- Use one consistent ITT domain for a logical workload; avoid marker spam.

LIMITATIONS / CAVEATS

- ITT regions only exist where you explicitly add them.
- ITT tasks measure host scopes; asynchronous GPU work needs device-track correlation.
- Instrumentation and full-rank capture have overhead—measure and scope them deliberately.
- `--checkpoint` replays layer work during Backward; the trace is telling the truth.

Three takeaways

- **Name Triton specializations** so kernel variants are identifiable in traces and profilers.
- **Add ITT regions** so model execution is visible as semantic phases instead of unlabeled activity.
- **Capture the run with THAPI/iprof** so those labels become a usable, time-aligned timeline in Perfetto.

Name kernels. Mark phases. Correlate layers. Then test a performance hypothesis.

Exercises, validated commands, expected output, and fallback traces: [HANDS_ON.md](#)